



Trabajo N°3

Franco Aguilar[†], Ivan Gallardo[†] y Diego Peralta[†]

[†]Universidad de Magallanes

Este informe fue compilado el 23 de junio de 2026

■ Índice

1	Introducción	2
2	Objetivos	3
3	Desarrollo	4
3.1	Algoritmos de Programación Dinámica (PD)	4
3.2	Algoritmos Greedy	6
3.3	Análisis Experimental	9
4	Conclusiones	17
5	Contribución por integrante	18

1. Introducción

En este proyecto se incorpora al sistema de deportistas el atributo costo, lo que permite modelar la creación de equipos bajo un presupuesto previo. A partir de esta modificación, el objetivo es seleccionar deportistas de manera que tengamos el mayor puntaje total sin superar el presupuesto disponible, además de mantener funcionalidades propias del sistema original, como generación de datos, ranking y búsqueda de deportistas.

Para apoyar estas funcionalidades, el sistema utiliza Quick Sort como algoritmo de ordenamiento. Este se emplea para ordenar deportistas según distintos criterios, como ID, puntaje, costo y la razón puntaje/costo. Quick Sort es utilizado para mostrar rankings tanto como para preparar las estrategias Greedy, donde el orden de selección depende directamente del criterio elegido. Además el proyecto utiliza búsqueda binaria para encontrar deportistas por ID, lo que requiere ordenar previamente los datos por identificador.

Sobre la base y las respectivas modificaciones, se implementaron soluciones mediante programación dinámica, usando tabulación y memoización, y mediante algoritmos greedy, considerando estrategias como mayor puntaje, menor costo y mejor razón puntaje/costo. Finalmente, el sistema permite ejecutar estos métodos desde consola y generar resultados experimentales en formato CSV, facilitando la comparación entre tiempos de ejecución, calidad de solución, costo utilizado y cercanía de los métodos greedy respecto a los resultados obtenidos con programación dinámica.

2. Objetivos

Implementar y comparar algoritmos de programación dinámica y algoritmos greedy para resolver el problema de selección de deportistas, considerando diversos escenarios teniendo en cuenta un presupuesto o no, integrando estas soluciones al sistema desarrollado en la tarea anterior.

1. Incorporar el atributo costo al sistema de deportistas y modelar la selección con presupuesto como un problema de optimización.
2. Implementar soluciones mediante Programación Dinámica, utilizando tabulación y memoización, para obtener una selección óptima de deportistas bajo un presupuesto.
3. Implementar Algoritmos Greedy para la selección de deportistas, considerando criterios como mayor puntaje, menor costo y mejor razón puntaje/costo, además de una selección sin presupuesto.
4. Evaluar y comparar experimentalmente los algoritmos implementados, considerando tiempos de ejecución, calidad de solución, uso del presupuesto y complejidad temporal y espacial.

3. Desarrollo

Antes de aplicar los algoritmos, se incorporó al modelo de deportistas el atributo costo. Este valor se generó como un entero entre 10 y 100. Se eligió este rango porque evita costos nulos, mantiene diferencias suficientes entre deportistas y permite construir presupuestos proporcionales al costo total sin que la tabla de programación dinámica crezca de forma descontrolada.

La magnitud del costo es especialmente importante para programación dinámica, ya que el tamaño de la tabla depende del presupuesto máximo W . Si los costos fueran mucho más altos, los presupuestos evaluados también crecerían y la memoria requerida aumentaría considerablemente. En cambio, en los algoritmos greedy el rango de costos no modifica la complejidad temporal, pero sí puede afectar la calidad de la selección, porque influye en qué deportistas caben dentro del presupuesto y en la razón puntaje/costo.

3.1. Algoritmos de Programación Dinámica (PD)

Problema a resolver

En esta parte del proyecto, el objetivo es formar un equipo de deportistas sin superar un presupuesto máximo. Cada deportista tiene dos datos importantes para esta selección: su puntaje, que representa qué tan conveniente es elegirlo, y su costo, que representa cuánto presupuesto consume.

Este problema se puede entender como una mochila 0/1. La 'mochila' sería el presupuesto disponible, y los 'objetos' serían los deportistas. Cada deportista puede entrar o no entrar al equipo, pero no puede elegirse más de una vez ni elegirse parcialmente. Por eso se llama mochila 0/1, para cada deportista solo existen dos opciones, seleccionarlo o no seleccionarlo.

Entonces, la meta es escoger la combinación de deportistas que entregue el mayor puntaje total posible, pero cuidando que la suma de sus costos no supere el presupuesto ingresado. Esta forma de ver el problema permite aplicar programación dinámica, ya que se pueden evaluar distintas decisiones y guardar las mejores soluciones parciales hasta llegar al mejor equipo posible.

Como se guarda la mejor solución

Para aplicar programación dinámica se utiliza una tabla que almacena las mejores soluciones parciales del problema. En esta tabla, las filas representan la cantidad de deportistas que se han considerado hasta cierto punto, mientras que las columnas representan los distintos presupuestos posibles. Cada casilla guarda el mejor puntaje que se puede alcanzar con esos deportistas y con ese presupuesto disponible.

La decisión principal consiste en analizar, para cada deportista, si conviene incluirlo o no en el equipo. Si el costo del deportista supera el presupuesto disponible, no puede ser seleccionado y se mantiene la mejor solución obtenida anteriormente. En cambio, si el deportista sí puede ser incluido, el algoritmo compara dos alternativas: dejarlo fuera del equipo o incorporarlo y sumar su puntaje al resultado previamente calculado con el presupuesto restante.

De esta forma, la tabla se va completando paso a paso hasta obtener la mejor combinación posible de deportistas. El resultado final corresponde al mayor puntaje alcanzable sin superar el presupuesto ingresado. Esta definición permite evitar cálculos repetidos y garantiza que se evalúen todas las combinaciones necesarias para encontrar una solución óptima.

Implementación por tabulación

La implementación por tabulación (bottom-up) utiliza un enfoque ascendente, donde el problema se resuelve construyendo primero soluciones pequeñas hasta llegar a la solución completa. Para esto se crea una tabla en la que las filas representan la cantidad de deportistas considerados y las columnas representan los distintos presupuestos posibles. Cada posición de la tabla almacena el mejor puntaje que se puede obtener con una cierta cantidad de deportistas y un presupuesto determinado.

El algoritmo recorre cada deportista y evalúa si puede ser incorporado al equipo según el presupuesto disponible en ese momento. Cuando el costo del deportista supera el presupuesto evaluado, se mantiene la mejor solución encontrada anteriormente. En cambio, si el deportista puede ser seleccionado, se comparan dos alternativas: conservar el resultado anterior o incluir al deportista actual sumando su puntaje. La tabla guarda siempre la alternativa que entrega el mayor puntaje.

Al finalizar el llenado de la tabla, la última posición contiene el puntaje óptimo que se puede alcanzar utilizando todos los deportistas disponibles sin superar el presupuesto máximo. Luego, a partir de los valores almacenados, se reconstruye el equipo seleccionado revisando qué deportistas fueron necesarios para alcanzar dicho resultado. Esta implementación garantiza una solución óptima, aunque su principal desventaja es que requiere más memoria a medida que aumentan la cantidad de deportistas y el presupuesto.

Elemento	Descripción
Filas	Representan la cantidad de deportistas considerados.
Columnas	Representan los posibles valores de presupuesto.
Valor almacenado	Mejor puntaje alcanzable para una cantidad de deportistas y presupuesto específico.

Cuadro 1. Elementos principales de la tabulación

Implementación por memoización

La implementación por memoización (top-down) utiliza un enfoque descendente, donde el problema se resuelve a partir del caso general y se divide en subproblemas más pequeños. En este caso, el algoritmo analiza cada deportista y el presupuesto disponible, decidiendo si conviene incluirlo o no en el equipo. A diferencia de la tabulación, no se llena toda la tabla de forma secuencial, sino que se calculan solo los estados que son necesarios durante las llamadas recursivas.

Para evitar repetir cálculos, se utiliza una tabla auxiliar donde se guardan los resultados ya obtenidos. Cuando el algoritmo necesita resolver un subproblema, primero revisa si ese resultado ya fue calculado previamente. Si existe, lo reutiliza directamente; si no existe, realiza la comparación entre incluir o no incluir al deportista actual y almacena el mejor resultado en la tabla. Esto permite reducir el trabajo repetido y mantener la misma lógica de decisión usada en el problema de mochila 0/1.

Al finalizar, la memoización entrega el mejor puntaje posible para todos los deportistas y el presupuesto máximo ingresado. Luego, al igual que en tabulación, se reconstruye el equipo seleccionado revisando las decisiones que llevaron al resultado óptimo. Este enfoque también garantiza una solución óptima, pero puede presentar un mayor costo por el uso de recursión, además de requerir una tabla de memoria para almacenar los subproblemas resueltos.

Elemento	Descripción
Enfoque	Descendente, parte del problema completo y lo divide en subproblemas.
Tabla auxiliar	Guarda los resultados ya calculados para evitar repetir operaciones.
Llamadas recursivas	Evalúan si conviene seleccionar o no seleccionar cada deportista.

Cuadro 2. Elementos principales de la memoización

Reconstrucción de la solución y complejidad

Una vez obtenido el puntaje óptimo mediante programación dinámica, es necesario reconstruir qué deportistas forman parte del equipo seleccionado. Para esto se revisa la tabla desde la última posición, es decir, desde el total de deportistas y el presupuesto máximo. Si el valor actual es distinto al

valor de la fila anterior, significa que el deportista correspondiente fue incluido en la solución. En ese caso, se agrega al equipo y se descuenta su costo del presupuesto restante.

Este proceso se realiza avanzando hacia atrás sobre los deportistas considerados. En tabulación, se revisan los valores almacenados en la tabla para identificar cuándo un deportista fue parte de la solución. En memoización, se comparan las alternativas de incluir o no incluir al deportista utilizando los resultados guardados en la memoria. De esta forma, no solo se obtiene el puntaje máximo, sino también la lista concreta de deportistas seleccionados, junto con su costo total y puntaje acumulado.

Sea n la cantidad de deportistas disponibles y W el presupuesto máximo. En programación dinámica, el estado del problema se define por dos variables: la cantidad de deportistas considerados y el presupuesto disponible. Por esta razón, la tabla utilizada tiene $(n + 1)(W + 1)$ posiciones.

En tabulación, cada posición de la tabla se calcula una sola vez, comparando la alternativa de no incluir al deportista actual con la alternativa de incluirlo si su costo lo permite. Por lo tanto, la complejidad temporal de tabulación es $O(nW)$. Además, como se almacena una tabla completa de tamaño $(n + 1)(W + 1)$, la complejidad espacial también es $O(nW)$.

En memoización, el estado es el mismo: índice del deportista y presupuesto restante. En el peor caso, se pueden llegar a calcular todos los estados posibles, por lo que la complejidad temporal también es $O(nW)$. La memoria utilizada para almacenar los resultados ya calculados es $O(nW)$, y adicionalmente se usa pila de recursión con profundidad máxima $O(n)$.

La reconstrucción de la solución recorre los deportistas desde el final hacia el inicio para identificar cuáles fueron seleccionados. Este proceso tiene costo $O(n)$, por lo que no cambia la complejidad total del algoritmo, que sigue dominada por $O(nW)$.

Implementación	Tiempo	Espacio	Observación
Tabulación	$O(nW)$	$O(nW)$	Llena la tabla de forma iterativa.
Memoización	$O(nW)$	$O(nW) + O(n)$	Calcula estados bajo demanda y usa recursión.
Reconstrucción	$O(n)$	$O(1)$ adicional	Recorre la solución desde la tabla.

Cuadro 3. Complejidad y características de la programación dinámica

3.2. Algoritmos Greedy

Problema a resolver

Los algoritmos greedy implementados abordan dos variantes del problema de selección de deportistas. La primera variante, con restricción de presupuesto, busca maximizar el puntaje total del equipo sin superar un presupuesto máximo W , sin fijar antes cuántos deportistas se incluirán. La segunda variante, sin restricción de presupuesto, selecciona exactamente k deportistas buscando el mayor puntaje posible, sin considerar los costos como limitación.

A diferencia de la programación dinámica, un algoritmo greedy no evalúa todas las combinaciones posibles. En cada paso toma la decisión localmente más conveniente según un criterio de prioridad fijo, sin reconsiderar las elecciones ya realizadas. Esto lo hace significativamente más eficiente en tiempo y memoria, pero no garantiza encontrar la solución óptima.

Estructura general de la implementación

Las tres estrategias con restricción de presupuesto comparten una función base denominada `greedy_generico`, que recibe como parámetros el criterio de ordenamiento (`SortCriteria`) y el orden (`SortOrder`) a utilizar. Esto permite reutilizar la misma lógica central para los tres criterios, variando únicamente cómo se ordena el arreglo antes del recorrido de selección. La firma de esta función es:

```
int greedy_generico(Deportista *deportistas, int cantidad, int presupuesto,
```

```
SortCriteria criteria, SortOrder order,
Deportista *seleccionados, int *costo_total,
float *puntaje_total);
```

El algoritmo funciona de la siguiente manera:

1. Se validan los parámetros de entrada. Si el arreglo es `NULL`, la cantidad es menor o igual a cero, o el presupuesto es inválido, se retorna inmediatamente con cero deportistas seleccionados.
2. Se crea una copia del arreglo original de deportistas con `malloc` y `memcpy`, para no alterar los datos de entrada.
3. Se ordena la copia usando `quick_sort_deportistas` según el criterio y orden indicados, utilizando pivote en el último elemento (`PIVOT_LAST`).
4. Se recorre el arreglo ordenado de izquierda a derecha. Para cada deportista, se verifica si su costo sumado al acumulado no supera el presupuesto. Si cabe, se agrega al arreglo de salida `seleccionados` y se acumulan su costo y puntaje. Si no cabe, se omite y se avanza al siguiente sin retroceder.
5. Al finalizar el recorrido, se libera la memoria de la copia con `free` y se retorna la cantidad de deportistas seleccionados.

Este comportamiento de tomar decisiones en base a una decisión inicial seleccionada es la base de un algoritmo greedy.

Criterio 1: Mayor puntaje primero

La función `greedy_por_puntaje` invoca a `greedy_generico` con el criterio `SORT_BY_PUNTAJE` en orden `DESCENDING`. El arreglo se ordena de mayor a menor puntaje, de modo que el algoritmo intenta incorporar primero a los deportistas más valiosos en términos de puntaje absoluto, independientemente de su costo.

```
int greedy_por_puntaje(Deportista *deportistas, int cantidad, int presupuesto,
    Deportista *seleccionados, int *costo_total,
    float *puntaje_total)
{
    return greedy_generico(deportistas, cantidad, presupuesto,
        SORT_BY_PUNTAJE, DESCENDING,
        seleccionados, costo_total, puntaje_total);
}
```

Esta estrategia tiende a producir equipos con puntajes altos cuando los deportistas más valiosos tienen costos razonables. Sin embargo, puede desperdiciar presupuesto si los primeros seleccionados son muy costosos, dejando poco margen para incluir a otros deportistas con puntajes menores que sí cabrían en el presupuesto restante.

Criterio 2: Menor costo primero

La función `greedy_por_costo` invoca a `greedy_generico` con el criterio `SORT_BY_COSTO` en orden `ASCENDING`. El arreglo se ordena de menor a mayor costo, priorizando a los deportistas más económicos para maximizar la cantidad de integrantes del equipo dentro del presupuesto.

```
int greedy_por_costo(Deportista *deportistas, int cantidad, int presupuesto,
    Deportista *seleccionados, int *costo_total,
```

```

        float *puntaje_total)
{
    return greedy_generico(deportistas, cantidad, presupuesto,
                           SORT_BY_COSTO, ASCENDING,
                           seleccionados, costo_total, puntaje_total);
}

```

Esta estrategia tiende a seleccionar la mayor cantidad posible de deportistas, ya que incorpora primero los más baratos. No obstante, no considera el valor de cada deportista, por lo que puede incluir deportistas con puntajes muy bajos mientras descarta opciones más rentables.

Criterio 3: Mejor razón puntaje/costo primero

La función `greedy_por_razon` invoca a `greedy_generico` con el criterio `SORT_BY_RATIO` en orden `DESCENDING`. El arreglo se ordena por la razón $\frac{\text{puntaje}}{\text{costo}}$ de mayor a menor, priorizando a los deportistas que entregan más puntaje por unidad de presupuesto consumida. En la comparación, si el costo de un deportista es cero, se utiliza directamente su puntaje como razón para evitar una división por cero.

```

int greedy_por_razon(Deportista *deportistas, int cantidad, int presupuesto,
                    Deportista *seleccionados, int *costo_total,
                    float *puntaje_total)
{
    return greedy_generico(deportistas, cantidad, presupuesto,
                           SORT_BY_RATIO, DESCENDING,
                           seleccionados, costo_total, puntaje_total);
}

```

De las tres estrategias con presupuesto, esta suele obtener resultados más cercanos al óptimo de la programación dinámica, ya que considera simultáneamente el valor y el costo de cada deportista. Sin embargo, tampoco garantiza la solución óptima, especialmente cuando el presupuesto restante no alcanza para el siguiente candidato más eficiente pero sí para una combinación de candidatos menos eficientes.

Criterio 4: Sin restricción de presupuesto

La función `greedy_sin_presupuesto` resuelve una variante distinta del problema: dado un valor k , selecciona exactamente los k deportistas con mayor puntaje, sin considerar ninguna restricción de presupuesto.

```

int greedy_sin_presupuesto(Deportista *deportistas, int cantidad, int k,
                          Deportista *seleccionados, int *costo_total,
                          float *puntaje_total);

```

Su funcionamiento es el siguiente: primero valida los parámetros y ajusta k si supera la cantidad total de deportistas disponibles. Luego crea una copia del arreglo y la ordena por `SORT_BY_PUNTAJE` en orden `DESCENDING`. Finalmente, toma directamente los primeros k elementos del arreglo ordenado, acumulando su costo y puntaje totales. A diferencia de las estrategias con presupuesto, este algoritmo no evalúa ninguna condición de presupuesto durante el recorrido: siempre selecciona exactamente k deportistas.

Esta estrategia es óptima para su variante del problema, ya que al ordenar por puntaje descendente y tomar los primeros k elementos garantiza que el equipo resultante tiene el mayor puntaje posible para ese tamaño de equipo.

Función	Criterio	Descripción
<code>greedy_por_puntaje</code>	Puntaje desc.	Prioriza deportistas con mayor puntaje absoluto.
<code>greedy_por_costo</code>	Costo asc.	Prioriza deportistas más económicos para maximizar cantidad.
<code>greedy_por_razon</code>	Razón puntaje/costo desc.	Prioriza deportistas con mejor eficiencia de valor por costo.
<code>greedy_sin_presupuesto</code>	Puntaje desc., toma k fijos	Selecciona exactamente k deportistas de mayor puntaje, sin restricción de presupuesto.

Cuadro 4. Resumen de los cuatro algoritmos greedy implementados

Reconstrucción de la solución

A diferencia de la programación dinámica, los algoritmos greedy no requieren una fase separada de reconstrucción del equipo. El conjunto seleccionado se construye directamente durante el recorrido del arreglo ordenado: cada deportista que cumple la condición de selección se agrega de inmediato al arreglo de salida `seleccionados`. Al finalizar el recorrido, dicho arreglo ya contiene el equipo completo, junto con el costo total acumulado (`costo_total`) y el puntaje total obtenido (`puntaje_total`), ambos calculados de forma incremental durante la selección.

Complejidad y limitaciones

La complejidad de las cuatro estrategias está dominada por el ordenamiento previo. Dado que se utiliza `quick_sort_deportistas` con partición de Lomuto y pivote en el último elemento, el costo en tiempo promedio es $O(n \log n)$, donde n es la cantidad de deportistas. El recorrido posterior de selección es lineal $O(n)$ y no altera el orden de complejidad total. En cuanto a memoria, solo se requiere una copia del arreglo de entrada de tamaño n , resultando en una complejidad espacial de $O(n)$, considerablemente menor que la tabla $(n + 1) \times (W + 1)$ que requiere la programación dinámica.

Aspecto	Descripción
Tiempo	$O(n \log n)$ en promedio, dominado por el ordenamiento con Quick Sort.
Espacio	$O(n)$, solo se requiere una copia del arreglo de entrada.
Ventaja	Eficiente en tiempo y memoria; no requiere tabla de subproblemas.
Limitación	Las estrategias con presupuesto no garantizan la solución óptima; pueden dejar presupuesto sin aprovechar de forma eficiente.

Cuadro 5. Complejidad y características de los algoritmos greedy

3.3. Análisis Experimental

La experimentación se realizó utilizando distintos tamaños de entrada generados a partir del archivo CSV de deportistas. En esta ejecución se evaluaron subconjuntos de 20, 100, 200, 500, 1000, 1500 y 2000 deportistas. Para cada tamaño de entrada se consideraron tres niveles de presupuesto relativo: 25 %, 50 % y 75 % del costo total de los deportistas considerados.

En el escenario con restricción de presupuesto se compararon cinco implementaciones: programación dinámica por tabulación, programación dinámica por memoización, greedy por mayor puntaje, greedy por menor costo y greedy por razón puntaje/costo. La programación dinámica se utilizó como referencia óptima, ya que evalúa sistemáticamente las combinaciones posibles sin superar el presupuesto máximo. Por ello, la calidad

de las estrategias greedy fue medida comparando su puntaje contra el puntaje obtenido por programación dinámica.

Además, se evaluó un escenario sin restricción de presupuesto mediante una estrategia greedy que selecciona los k deportistas con mayor puntaje. En este caso no existe límite de costo, por lo que ordenar por puntaje descendente y escoger los mejores candidatos constituye una solución directa para el problema planteado.

Tamaños de entrada y presupuestos evaluados

La Tabla 6 muestra los presupuestos utilizados para cada tamaño de entrada. Estos presupuestos fueron calculados como una proporción del costo total de los n deportistas considerados.

Cuadro 6. Tamaños de entrada y presupuestos evaluados

n	25 %	50 %	75 %
20	266	533	800
100	1461	2923	4384
200	2737	5475	8213
500	6847	13694	20541
1000	13569	27139	40709
1500	20427	40854	61281
2000	27465	54931	82396

El crecimiento de los presupuestos es relevante porque afecta directamente a la programación dinámica. En esta formulación, la tabla de programación dinámica depende tanto de la cantidad de deportistas n como del presupuesto W . Por lo tanto, al aumentar el tamaño de entrada y el presupuesto, se incrementa tanto el tiempo de ejecución como el uso de memoria requerido por el algoritmo.

Comparación entre programación dinámica por tabulación y memoización

Las dos implementaciones de programación dinámica entregaron el mismo puntaje óptimo en todos los casos evaluados. Esto permite validar que ambas formulaciones resuelven correctamente el mismo problema de optimización. Sin embargo, sus tiempos de ejecución presentan diferencias claras.

La Tabla 7 muestra el tiempo promedio de ejecución de tabulación y memoización para cada tamaño de entrada. Para cada n , el valor corresponde al promedio de los tres niveles de presupuesto.

Cuadro 7. Tiempo promedio de ejecución de programación dinámica

n	Tabulación (s)	Memoización (s)	Memo/Tab
20	0.000079	0.000123	1.56
100	0.002249	0.004449	1.98
200	0.007743	0.016311	2.11
500	0.049565	0.124788	2.52
1000	0.197553	0.521364	2.64
1500	0.427972	1.158755	2.71
2000	0.776768	2.129487	2.74

Los resultados muestran que la memoización fue consistentemente más lenta que la tabulación. Para $n = 20$, la memoización tardó aproximadamente 1.56 veces más. Sin embargo, a medida que crece el tamaño de entrada, esta diferencia aumenta. Para $n = 2000$, la memoización tardó en promedio 2.74 veces más que la tabulación.

Esto se explica porque, aunque la memoización evita recalcular subproblemas ya resueltos, en este problema una gran parte de los estados termina siendo necesaria para obtener la solución óptima. Por lo tanto, la memoización no logra reducir suficientemente el espacio de cálculo y además incorpora el costo adicional de las llamadas recursivas. La tabulación, en cambio, construye la solución de forma iterativa, evitando ese sobre costo.

En consecuencia, para esta implementación, la tabulación resultó más eficiente que la memoización, manteniendo la misma calidad de solución.

Calidad general de las estrategias greedy

Para analizar la calidad de las soluciones greedy, se consideró el puntaje obtenido por programación dinámica como el 100 % de referencia. La calidad de cada estrategia greedy se calculó como:

$$\text{Calidad} = \frac{\text{Puntaje Greedy}}{\text{Puntaje PD}} \times 100$$

De forma complementaria, la diferencia porcentual respecto al óptimo puede interpretarse como:

$$\text{Diferencia} = 100 - \text{Calidad}$$

La Tabla 8 resume la calidad promedio, mínima y máxima de cada estrategia greedy considerando todos los tamaños de entrada y todos los presupuestos evaluados.

Cuadro 8. Calidad general de las estrategias greedy respecto a programación dinámica

Algoritmo	Calidad prom.	Calidad mín.	Calidad máx.	Dif. prom.
Greedy por puntaje	92.35 %	79.53 %	100.00 %	7.65 %
Greedy por costo	87.83 %	82.74 %	94.25 %	12.17 %
Greedy por puntaje/costo	99.86 %	98.63 %	100.00 %	0.14 %

La estrategia greedy por razón puntaje/costo fue claramente la más cercana a la solución óptima. Su calidad promedio fue de 99.86 %, con una diferencia promedio de solo 0.14 % respecto a programación dinámica. Esto indica que, para los datos evaluados, considerar simultáneamente el puntaje aportado por un deportista y el costo necesario para seleccionarlo produce soluciones muy competitivas.

Greedy por mayor puntaje obtuvo una calidad promedio de 92.35 %. Si bien esta estrategia puede comportarse bien cuando el presupuesto es amplio, presenta una pérdida importante cuando el presupuesto es restrictivo. Esto ocurre porque prioriza deportistas con alto puntaje sin considerar suficientemente su costo, lo que puede consumir rápidamente el presupuesto y dejar fuera combinaciones más eficientes.

Greedy por menor costo obtuvo una calidad promedio de 87.83 %, siendo la estrategia menos efectiva en términos de puntaje total. Aunque suele seleccionar una mayor cantidad de deportistas, esto no implica necesariamente una mejor solución, ya que el objetivo del problema es maximizar el puntaje total y no la cantidad de seleccionados.

Efecto del presupuesto en la calidad de greedy

La Tabla 9 muestra la calidad promedio de cada estrategia greedy según el presupuesto relativo disponible.

Cuadro 9. Calidad promedio de greedy según presupuesto disponible

Algoritmo	25 %	50 %	75 %	Promedio
Greedy por puntaje	82.68 %	95.23 %	99.15 %	92.35 %
Greedy por costo	85.57 %	86.84 %	91.10 %	87.83 %
Greedy por puntaje/costo	99.72 %	99.90 %	99.97 %	99.86 %

El efecto del presupuesto es especialmente notorio en la estrategia greedy por puntaje. Con un presupuesto de 25 %, su calidad promedio fue de 82.68 %, mientras que con 75 % de presupuesto subió a 99.15 %. Esto evidencia que esta estrategia depende fuertemente de cuán restrictivo sea el presupuesto: cuando hay poco presupuesto, escoger primero los deportistas de mayor puntaje puede ser una mala decisión si esos deportistas son costosos.

La estrategia greedy por costo mejora levemente al aumentar el presupuesto, pero se mantiene por debajo de las otras estrategias. Incluso con 75 % de presupuesto, su calidad promedio fue de 91.10 %. Esto indica que escoger deportistas baratos permite conformar equipos más numerosos, pero no necesariamente equipos con mayor puntaje acumulado.

Por otro lado, greedy por razón puntaje/costo mantiene una calidad muy alta en todos los niveles de presupuesto. Su peor promedio se observa con 25 % de presupuesto, pero aun así alcanza 99.72 %. Esto muestra que es la estrategia más robusta frente a cambios en la restricción presupuestaria.

Calidad promedio según tamaño de entrada

La Tabla 10 muestra la calidad promedio de cada estrategia greedy agrupada por tamaño de entrada, promediando los tres presupuestos evaluados.

Cuadro 10. Calidad promedio de greedy según tamaño de entrada			
<i>n</i>	Greedy puntaje	Greedy costo	Greedy puntaje/costo
20	97.17 %	93.03 %	99.37 %
100	90.89 %	84.82 %	99.78 %
200	91.96 %	87.44 %	99.93 %
500	92.57 %	87.71 %	99.99 %
1000	91.44 %	87.57 %	99.99 %
1500	91.09 %	87.46 %	99.99 %
2000	91.34 %	86.81 %	100.00 %

La calidad de greedy por razón puntaje/costo se mantiene prácticamente constante y cercana al óptimo en todos los tamaños de entrada. Esto indica que, para los datos experimentales utilizados, esta estrategia escala bien en calidad incluso cuando el número de deportistas aumenta hasta 2000.

En cambio, greedy por puntaje y greedy por costo presentan una calidad menor y más variable. Greedy por puntaje se ve especialmente afectado cuando el presupuesto es bajo, mientras que greedy por costo mantiene una calidad inferior porque prioriza deportistas baratos sin considerar suficientemente el puntaje que aportan.

Caso representativo con $n = 2000$

Para observar el comportamiento de los algoritmos en el mayor tamaño de entrada evaluado, la Tabla 11 presenta los resultados obtenidos para $n = 2000$ en los tres niveles de presupuesto.

Cuadro 11. Resultados detallados para $n = 2000$

Presupuesto	Algoritmo	Tiempo (s)	Puntaje	Diferencia	Calidad	Seleccionados
25 %	PD Tabulación	0.402599	55124.59	0.00	100.00 %	773
25 %	PD Memoización	1.281060	55124.59	0.00	100.00 %	773
25 %	Greedy puntaje	0.000312	44599.22	10525.38	80.91 %	506
25 %	Greedy costo	0.000434	46298.91	8825.68	83.99 %	904
25 %	Greedy puntaje/costo	0.000393	55120.03	4.56	99.99 %	774
50 %	PD Tabulación	0.765809	81703.68	0.00	100.00 %	1185
50 %	PD Memoización	2.231556	81703.68	0.00	100.00 %	1185
50 %	Greedy puntaje	0.000324	76944.91	4758.77	94.18 %	1004
50 %	Greedy costo	0.000439	69919.68	11784.00	85.58 %	1353
50 %	Greedy puntaje/costo	0.000394	81701.70	1.98	100.00 %	1186
75 %	PD Tabulación	1.161895	97415.48	0.00	100.00 %	1596
75 %	PD Memoización	2.875845	97415.48	0.00	100.00 %	1596
75 %	Greedy puntaje	0.000326	96371.97	1043.51	98.93 %	1511
75 %	Greedy costo	0.000425	88505.93	8909.55	90.85 %	1704
75 %	Greedy puntaje/costo	0.000387	97414.60	0.88	100.00 %	1596

Este caso permite observar varias conclusiones importantes. Primero, programación dinámica por tabulación y memoización entregan el mismo puntaje en todos los presupuestos, confirmando la concordancia entre ambos enfoques. Sin embargo, memoización tarda considerablemente más. Para $n = 2000$ y presupuesto de 75 %, la tabulación tardó 1.161895 segundos, mientras que la memoización tardó 2.875845 segundos.

Segundo, las estrategias greedy fueron muchísimo más rápidas que programación dinámica. En el presupuesto de 75 %, greedy por razón puntaje/costo tardó solo 0.000387 segundos y obtuvo un puntaje de 97414.60, frente al óptimo de 97415.48. La diferencia fue de apenas 0.88 puntos, lo que representa una pérdida prácticamente nula en términos porcentuales.

Tercero, se observa que greedy por costo puede seleccionar más deportistas que programación dinámica sin obtener un mejor puntaje. Con $n = 2000$ y presupuesto de 75 %, greedy por costo seleccionó 1704 deportistas, mientras que programación dinámica seleccionó 1596. Sin embargo, el puntaje de greedy por costo fue 88505.93, inferior al óptimo de 97415.48. Esto demuestra que maximizar la cantidad de seleccionados no equivale a maximizar el puntaje total.

Finalmente, greedy por puntaje mejora a medida que el presupuesto aumenta. Con 25 % de presupuesto obtuvo solo 80.91 % de calidad, pero con 75 % alcanzó 98.93 %. Esto confirma que su principal debilidad aparece cuando el presupuesto es restrictivo.

Comparación de tiempos: aceleración de greedy frente a programación dinámica

La Tabla 12 muestra cuántas veces más rápida fue la estrategia greedy por razón puntaje/costo respecto a programación dinámica por tabulación. Se utiliza esta estrategia porque fue la greedy con mejor calidad general.

Cuadro 12. Aceleración de greedy puntaje/costo respecto a PD por tabulación

n	25 %	50 %	75 %
20	24.4x	58.1x	78.8x
100	64.4x	253.9x	306.7x
200	164.3x	347.0x	520.0x
500	325.7x	666.8x	1080.5x
1000	595.4x	1167.2x	1649.2x
1500	797.1x	1582.6x	2245.4x
2000	1023.4x	1943.2x	3000.8x

Los resultados muestran que la diferencia temporal entre greedy y programación dinámica aumenta con el tamaño de entrada y con el presupuesto. Para $n = 2000$ y presupuesto de 75 %, greedy por razón puntaje/costo fue aproximadamente 3000 veces más rápido que programación dinámica por tabulación, manteniendo una calidad prácticamente igual al óptimo.

Esto evidencia una diferencia fundamental entre ambos paradigmas. Programación dinámica entrega garantías de optimalidad, pero su costo crece de forma importante al aumentar n y W . Greedy, en cambio, no explora todo el espacio de soluciones, por lo que obtiene resultados en tiempos significativamente menores. Su desventaja es que no garantiza optimalidad para todos los casos, aunque en este experimento la estrategia puntaje/costo obtuvo resultados muy cercanos al óptimo.

Uso de memoria estimado en programación dinámica

Además del tiempo de ejecución, se estimó el uso de memoria de la tabla de programación dinámica. Para esta formulación del problema, la cantidad de celdas de la tabla puede aproximarse como:

$$(n + 1)(W + 1)$$

Si cada celda almacena un valor de tipo `float`, la memoria estimada se obtiene multiplicando la cantidad de celdas por `sizeof(float)`. La Tabla 13 presenta la memoria estimada en megabytes.

Cuadro 13. Memoria estimada de la tabla de programación dinámica

n	25 % (MB)	50 % (MB)	75 % (MB)
20	0.02	0.04	0.07
100	0.59	1.18	1.77
200	2.20	4.40	6.60
500	13.72	27.44	41.17
1000	54.33	108.67	163.00
1500	122.65	245.29	367.94
2000	219.84	439.68	659.51

La memoria estimada crece de forma directa con el tamaño de entrada y con el presupuesto. Para $n = 2000$, la tabla requiere aproximadamente 219.84 MB con presupuesto de 25 %, 439.68 MB con presupuesto de 50 % y 659.51 MB con presupuesto de 75 %.

Estos valores muestran que, aunque la programación dinámica todavía fue viable para $n = 2000$ en esta ejecución, su consumo de memoria ya es considerable. Si el número de deportistas, el rango de costos o el presupuesto aumentan, la tabla puede crecer hasta convertirse en una limitación práctica. Este comportamiento contrasta con los algoritmos greedy, que no requieren construir una tabla de estados y, por lo tanto, tienen un uso adicional de memoria mucho menor.

Escenario sin restricción de presupuesto

En el escenario sin presupuesto se evaluó una estrategia greedy que selecciona exactamente k deportistas con mayor puntaje. Para cada tamaño de entrada se utilizó $k = n/2$. La Tabla 14 muestra los resultados obtenidos.

Cuadro 14. Resultados de greedy sin restricción de presupuesto					
n	k	Tiempo (s)	Puntaje	Costo	Seleccionados
20	10	0.000001	593.58	402	10
100	50	0.000007	3635.65	3038	50
200	100	0.000018	7351.37	5542	100
500	250	0.000061	18636.13	13343	250
1000	500	0.000142	38204.93	27030	500
1500	750	0.000225	58045.80	40627	750
2000	1000	0.000302	76737.11	54788	1000

Los tiempos observados son extremadamente bajos incluso para $n = 2000$. Esto se debe a que, al no existir restricción de presupuesto, el problema se reduce a ordenar los deportistas por puntaje y seleccionar los k mejores. En este escenario, el criterio greedy es adecuado, ya que no existe una restricción adicional que pueda invalidar una elección local.

4. Conclusiones

A partir del análisis teórico y experimental realizado, se extraen las siguientes conclusiones respecto al comportamiento de los algoritmos estudiados.

- Las implementaciones de programación dinámica por tabulación y memoización entregaron el mismo puntaje óptimo en todos los casos evaluados, validando la concordancia entre ambos enfoques.
- La tabulación fue más eficiente que la memoización. Para $n = 2000$, la memoización tardó en promedio 2.74 veces más que la tabulación.
- Los algoritmos greedy fueron varios órdenes de magnitud más rápidos que programación dinámica. En el caso de $n = 2000$ y presupuesto de 75 %, greedy por razón puntaje/costo fue aproximadamente 3000 veces más rápido que la tabulación.
- La calidad de las soluciones greedy depende fuertemente del criterio utilizado. Greedy por razón puntaje/costo fue la estrategia más cercana al óptimo, con una calidad promedio de 99.86 %.
- Greedy por puntaje mejora cuando el presupuesto es más amplio, pero pierde calidad bajo presupuestos restrictivos. Su calidad promedio fue de 82.68 % con presupuesto de 25 % y de 99.15 % con presupuesto de 75 %.
- Greedy por costo fue la estrategia menos efectiva en términos de puntaje total. Aunque puede seleccionar más deportistas, no necesariamente maximiza la ganancia.
- La programación dinámica garantiza optimalidad, pero su uso de memoria crece con n y W . Para $n = 2000$ y presupuesto de 75 %, la memoria estimada fue de aproximadamente 659.51 MB.
- En el escenario sin restricción de presupuesto, greedy por mayor puntaje resulta suficiente, ya que el problema se reduce a seleccionar los k deportistas con mayor puntaje.

En conclusión, la programación dinámica es preferible cuando se requiere garantizar la optimalidad de la solución y los recursos computacionales disponibles permiten asumir su costo temporal y espacial. Por otro lado, las estrategias greedy son convenientes cuando se necesita una solución rápida y suficientemente buena. Entre las estrategias evaluadas, greedy por razón puntaje/costo presentó el mejor equilibrio entre eficiencia temporal y calidad de solución.

Es importante señalar que, aunque greedy por razón puntaje/costo obtuvo resultados prácticamente óptimos en esta experimentación, esto no significa que garantice optimalidad en todos los casos posibles. Al tratarse de un algoritmo voraz, toma decisiones locales y no explora todas las combinaciones. Por lo tanto, pueden existir instancias donde su solución no coincida con la óptima obtenida por programación dinámica.

5. Contribución por integrante

A continuación se detalla la contribución realizada por cada uno de los integrantes del grupo en el desarrollo del proyecto.

Franco Aguilar

Adaptación de la generación de datos

Adaptó la generación de deportistas agregando un costo a cada jugador de manera aleatoria para implementar algoritmos de programación dinámica y algoritmos greedy, para su posterior análisis experimental.

Implementación de algoritmos de programación dinámica

- Tabulación (bottom-up)
- Memoización (top-down)

Iván Gallardo

Implementación de algoritmo Greedy sin presupuesto

Implementación de la funcionalidad para conformar equipos sin restricción presupuestaria mediante una estrategia greedy. Ordena los deportistas por puntaje descendente y selecciona exactamente los k mejores candidatos.

Análisis experimentales

Creación del benchmark experimental en `experiment.c`. Análisis e interpretación de los datos generados, ver sección 3.3

Diego Peralta

Implementación de algoritmos Greedy

- Criterio 1: Mayor puntaje primero
- Criterio 2: Menor costo primero
- Criterio 3: Mejor razón puntaje/costo primero